

Solving: Longest Substring Without Repeating Characters

Let's solve this step-by-step using the **sliding window technique**!

Understanding the Problem

We need to find: "abcabcbb" → "abc" (length 3)

Because:

- "abc" has no repeating characters
- Any longer substring would have repeats

Thought Process

Imagine you're reading a book and highlighting a passage:

- Start highlighting at word 1
- Keep highlighting until you see a word you already highlighted
- When you see a repeat, stop highlighting from that point
- Move the starting point past the repeated word
- Keep track of the **longest highlight** you made

Step-by-Step Algorithm

1. **Initialize:** Empty set to track characters in current window
2. **Two pointers:** `left` (window start) and `right` (window end)
3. **Expand:** Move `right` pointer, add new character to set
4. **If duplicate found:**
 - Move `left` pointer forward, removing characters from set
 - Until the duplicate is removed
5. **Track:** Maximum window size seen

Let's Code This Together!

Version 1: Basic & Readable

```
def longest_unique_substring(s: str) -> int:
    """
    Find the length of the longest substring without repeating characters.
```

Example:

Input: "abcabcbb" → Output: 3 ("abc")

Input: "bbbbbb" → Output: 1 ("b")

Input: "pwwkew" → Output: 3 ("wke")

"""

If string is empty, return 0

if not s:

return 0

Set to store characters in current window

char_set = set()

Pointers for sliding window

left = 0

max_length = 0

Slide the right pointer through the string

for right in range(len(s)):

If current character is already in the set (duplicate found)

while s[right] in char_set:

Remove leftmost character from set

char_set.remove(s[left])

Move left pointer forward

left += 1

Add current character to set

char_set.add(s[right])

Update max_length if current window is larger

current_length = right - left + 1

max_length = max(max_length, current_length)

return max_length

Version 2: With Visualization (For Learning)

```
def longest_unique_substring_visual(s: str) -> tuple:
```

```
    """
```

```
    Returns both length and the actual substring.
```

```
    """
```

```
    if not s:
```

```
        return 0, ""
```

```

char_set = set()
left = 0
max_length = 0
best_substring = ""

for right in range(len(s)):
    print(f"\nStep {right}: Character '{s[right]}'")
    print(f"Current window: {s[left:right+1]}")

    # Remove characters from left until no duplicate
    while s[right] in char_set:
        print(f"  Duplicate '{s[right]}' found! Removing '{s[left]}'"
from left")
        char_set.remove(s[left])
        left += 1

    # Add current character
    char_set.add(s[right])

    # Check if this is the longest substring so far
    current_length = right - left + 1
    if current_length > max_length:
        max_length = current_length
        best_substring = s[left:right+1]
        print(f"  New longest substring found: '{best_substring}'"
(length {max_length})")
    else:
        print(f"  Current window length: {current_length}")

return max_length, best_substring

```

Version 3: More Efficient (Using Dictionary)

```

def longest_unique_substring_optimal(s: str) -> int:
    """
    More efficient version using dictionary to store last seen positions.
    This avoids the inner while loop in some cases.
    """
    if not s:
        return 0

    # Dictionary to store last seen index of each character
    last_seen = {}

    left = 0

```

```

max_length = 0

for right in range(len(s)):
    current_char = s[right]

    # If we've seen this character before and it's within our current
window
    if current_char in last_seen and last_seen[current_char] >= left:
        # Jump left pointer to just after the last occurrence
        left = last_seen[current_char] + 1

    # Update last seen position
    last_seen[current_char] = right

    # Calculate current window length
    current_length = right - left + 1
    max_length = max(max_length, current_length)

return max_length

```

Let's Test Our Code!

```

# Test cases
test_cases = [
    ("abcabcbb", 3),      # "abc"
    ("bbbbbb", 1),       # "b"
    ("pwwkew", 3),       # "wke"
    ("", 0),              # empty string
    ("abcdef", 6),        # entire string
    ("aab", 2),           # "ab"
    ("dvdf", 3),          # "vdf"
]

print("Testing our function:")
print("=" * 40)

for test_str, expected in test_cases:
    result = longest_unique_substring(test_str)
    status = "✓" if result == expected else "✗"
    print(f"{status} '{test_str}' -> {result} (expected: {expected})")

# Let's see the visualization for one example
print("\n\nVisualizing for 'abcabcbb':")
print("=" * 40)

```

```
max_len, substring = longest_unique_substring_visual("abcabcbb")
print(f"\nFinal result: Length = {max_len}, Substring = '{substring}'")
```

Visual Walkthrough for "abcabcbb"

Let's trace through the algorithm:

```
String: a b c a b c b b
Step 0: Add 'a' → window: "a" (length 1)
Step 1: Add 'b' → window: "ab" (length 2)
Step 2: Add 'c' → window: "abc" (length 3) ← NEW MAX!

Step 3: Try to add 'a' → Duplicate found!
        Remove 'a' from left → window: "bca"
Step 4: Try to add 'b' → Duplicate found!
        Remove 'b' from left → window: "cab"
Step 5: Try to add 'c' → Duplicate found!
        Remove 'c' from left → window: "abc"
Step 6: Try to add 'b' → Duplicate found!
        Remove 'a' from left → window: "cb"
Step 7: Try to add 'b' → Duplicate found!
        Remove 'c' from left → window: "b"
```

Final answer: 3 (from substring "abc")

Complexity Analysis

- **Time Complexity:** $O(n)$ - Each character visited at most twice
- **Space Complexity:** $O(\min(n, m))$ where m is character set size
 - For ASCII: $O(1)$ since only 128 possible characters
 - For Unicode: $O(\min(n, 143,859))$ - all possible characters

Key Insights

1. **Why sliding window works here?**
 - We're looking for **consecutive characters** (substring)
 - When we see a repeat, we know we need to **shrink from the left**
 - We can **reuse** information about what's in our current window
2. **The set/dictionary is our "memory"** of what's in the window
3. **The two pointers define our window:**
 - `left` : Start of current unique substring
 - `right` : End of current unique substring



Common Mistakes to Avoid

```
# WRONG: This doesn't handle all cases properly
def wrong_solution(s):
    seen = set()
    max_len = 0
    for char in s:
        if char in seen:
            seen.clear() # WRONG: Clears everything, not just until
duplicate
            seen.add(char)
            max_len = max(max_len, len(seen))
    return max_len

# Why wrong? For "dvdf":
# - d → {"d"} length 1
# - v → {"d", "v"} length 2
# - d → clears everything, starts fresh {"d"} length 1
# - f → {"d", "f"} length 2
# Returns 2, but correct answer is 3 ("vdf")
```



Practice Problems to Try Next

1. **Minimum Window Substring** (Harder)
2. **Longest Substring with At Most K Distinct Characters**
3. **Longest Substring with At Most Two Distinct Characters**
4. **Find All Anagrams in a String**

? Quick Quiz

Q: What would our function return for "tmmzuxt"?

A: Let's think:

- "t" → "tm" → "tm" (duplicate 'm') → "m" → "mz" → "mzu" → "mzux" → "mzuxt"
- Longest: "mzuxt" (length 5)

Try running it to check!

Remember: The sliding window technique is perfect for substring problems because it lets us efficiently explore all possible substrings without starting over each time! 🧩✨

Want me to explain any part in more detail or walk through another example?